



Time: 3 hours

150 marks

1. This question paper consists of 15 pages. Please check that your question paper is complete.
2. This question paper is to be answered using object-oriented programming principles. Your program must make sensible use of methods and parameters.
3. This paper is divided into two sections. All candidates must answer both sections.
4. This paper is set in programming terms that are not specific to any particular programming language (Java/Delphi) or database (Access/MySQL/JavaDB).
5. Ensure that you answer the questions in the manner described because marks are awarded according to the specifications that are given in the question.
6. Only answer what is asked in each question. For example, if the question does not ask for data validation, no marks will be awarded for it; therefore, no code needs to be written for data validation.
7. If you cannot get a section of code to work, comment it out so that it will not be executed, and you can continue with the examination. If possible, try to explain the error to aid the marker.
8. Your programs must be coded so that they will work with any data, not just the sample data supplied or any data extracts that appear in the question paper. You are advised to look at the provided data files carefully.
9. Make sure that routines, such as searches and sorts for arrays, are developed from first principles and that you do not use the built-in features of a programming language for any of these routines.
10. All data structures must be defined and declared by you, the programmer. You may not use the interface's components to store and retrieve data.
11. Read the whole question paper before you choose a data structure. You may find that there could be an alternative method of representing the data that is more efficient considering the questions that are asked in the paper.



12. You must save all your work regularly on the disk you have been given or the disk space allocated to you for this examination. You must also create a backup of the original files before you start in case your solution accidentally modifies the original version.
13. If your examination is interrupted by a technical problem, such as a power failure, when you resume writing, you will only be given the time that remains when the interruption occurred, to complete your examination. No extra time will be given to catch up on work that was not saved.
14. Make sure that your examination number appears as a comment in every program you code and on every printed page you hand in.
15. Check every printed page you print, making sure you submit the code for every question.
16. Check that your printout has the correct indenting and formatting and has not truncated any text.
17. Check that each page is clearly legible with a font size of 12.
18. Print a code listing of all the programs/class files you code. Printing must be done after the examination. You will be given half an hour to print after the examination. Your teacher will tell you what arrangements have been made to print your work.
19. You should have the following two folders (in bold) and files. These files are to be used as data for this examination. Note that the database files are in MS Access, JavaDB, and MySQL formats and contain the same data in each file. You only need the file that matches the database application you will need for Section A. Ensure that you can open the files with the packages you will use to code your solutions to this examination.

Section A:

Theatre_Access.mdb
Theatre_JavaDB.sql
Theatre_MySQL.sql
SQLAnswerSheet.rtf
SQLBrowser.exe

Section B:

Production.txt

SCENARIO:**SECTION A SQL****QUESTION 1**

The Swift Theatre Company owns theatres in cities throughout South Africa and performs various plays in each. All information on theatres, plays, and ticket sales for one week in February is contained in the database, **Theatre**.

tblTheatres contains the details of all the theatres.

FIELDS	DATA TYPE	DESCRIPTION
TheatreID	Integer	A unique auto-numbered identification number for each theatre.
TName	Text	The name of the theatre.
City	Text	The city the theatre is in.
TicketPrice	Integer	The cost of a ticket in rands for the theatre.
NumStaff	Integer	The number of staff employed, not including actors, for the theatre.

The records in tblTheatres:

TheatreID	TName	City	TicketPrice	NumStaff
1	The Grand Globe	Johannesburg	410	17
2	Dramatic Acts	Cape Town	230	13
3	Community Playhouse	Durban	370	18
4	Theatre House	Johannesburg	190	12
5	Stage and Song	Cape Town	410	19
6	Dinner and Drama	Johannesburg	140	5
7	Grand Backstage	Gqeberha	360	16

tblPlays contains the details of the plays.

FIELDS	DATA TYPE	DESCRIPTION
PlayID	Integer	A unique auto-numbered identification number for each play.
PName	Text	The name of the play.
PubDom	Boolean	Whether the play is public domain or not.
Fee	Integer	The cost to license the play per showing.
NumActors	Integer	The number of actors in the play.

The records in tblPlays:

PlayID	PName	PubDom	Fee	NumActors
1	Frankenstein	Yes	0	11
2	Hello and Goodbye	No	2075	2
3	Peter Pan	Yes	0	32
4	Umabatha	No	1825	46
5	Have You Seen Zandile	No	2100	2
6	A Doll House	Yes	0	10
7	When Life Gives Lemons	No	1500	20

tblSales contains the details of all ticket sales from 4 February 2024 to 9 February 2024.

FIELDS	DATA TYPE	DESCRIPTION
TheatreID	Integer	The ID of the theatre the tickets are for. This is a foreign key to tblTheatres.
PlayID	Integer	The ID of the play the tickets are for. This is a foreign key to tblPlays.
ShowDate	Date	The date the tickets are for.
Tickets	Integer	The number of tickets sold.

The first 10 records of tblSales:

TheatreID	PlayID	ShowDate	Tickets
1	1	2024/02/04	249
1	1	2024/02/05	165
1	1	2024/02/06	255
1	2	2024/02/07	241
1	2	2024/02/08	306
2	2	2024/02/09	175
2	4	2024/02/07	232
2	4	2024/02/08	202
3	4	2024/02/06	252
5	3	2024/02/04	343

Note: Do not rename columns/use an alias unless explicitly told to do so in a question.

- 1.1 Display all details of plays that are in the public domain or with more than 30 actors. The correct output is shown below:

PlayID	PName	PubDom	Fee	NumActors
1	Frankenstein	Yes	0	11
3	Peter Pan	Yes	0	32
4	Umabatha	No	1825	46
6	A Doll House	Yes	0	10

(4)

- 1.2 Swift Theatre Company decides to award every theatre employee a bonus of between R500 and R1000 (inclusive). Calculate the total amount of money that must be sent to every theatre by multiplying the number of staff by a random number between 500 and 1000 (inclusive). Ensure that a different random number is generated for each record. Sort the theatres by name alphabetically. Display the theatre name and the bonus amount.

The sample output is shown below; your bonus may differ because of the random number generation.

TName	Bonus
Community Playhouse	5742
Dinner and Drama	1620
Dramatic Acts	1391
Grand Backstage	8192
Stage and Song	6840
The Grand Globe	3803
Theatre House	1476

(5)

- 1.3 The company needs a list of all theatres which employ large numbers of staff or have very low ticket prices, except for any theatres in Durban. Write a query that will display all the details about theatres that employ more than 15 staff or have a ticket price of less than R150. Ensure that no theatres from Durban are on the list. The correct output is shown below:

TheatreID	TName	City	TicketPrice	NumStaff
1	The Grand Globe	Johannesburg	410	17
5	Stage and Song	Cape Town	410	19
6	Dinner and Drama	Johannesburg	140	5
7	Grand Backstage	Gqeberha	360	16

(4)

- 1.4 Display the total number of staff for all theatres. The correct output is shown below:

TotalStaff
100

(3)

- 1.5 Write a query to determine how many plays are performed on each date at any theatre. Display the date and the number of plays for that date; name this column TotalPlays. The correct output is shown below:

ShowDate	TotalPlays
2024/02/04	3
2024/02/05	1
2024/02/06	4
2024/02/07	3
2024/02/08	2
2024/02/09	4

(4)

- 1.6 Display the play name and the dates it is performed for all plays. Sample output is given below:

Note only the first six rows are displayed.

PName	ShowDate
Frankenstein	2024/02/04
Frankenstein	2024/02/05
Frankenstein	2024/02/06
Hello and Goodbye	2024/02/09
Hello and Goodbye	2024/02/06
Hello and Goodbye	2024/02/07

(4)

- 1.7 Swift Theatre Company wants a hashtag that audience members can use when they see each play. Combine:

- The first letter of the city
- The last four letters of the city
- The word 'Feb'
- The day of the ticket

into one hashtag.

Sample output is shown below:

Note only the first six rows are displayed.

Hashtag
JburgFeb4
JburgFeb5
JburgFeb6
JburgFeb7
JburgFeb8
CTownFeb9

(6)

- 1.8 Display the name of all theatres that have not sold any tickets. The correct output is shown below:

TName
Theatre House

(4)

- 1.9 Display the theatre name, play name, and the average number of tickets sold for each. Only display theatres and plays where the average number of tickets sold is above 250. The correct output is shown below:

TName	PName	AvgTicketsSold
Community Playhouse	Umabatha	252
Grand Backstage	Have You Seen Zandile	321
Stage and Song	Peter Pan	297.5
The Grand Globe	Hello and Goodbye	273.5

(8)

- 1.10 The play 'When Life Gives Lemons' has been changed to be in the public domain, meaning the fee will now be 0. Write a query to change the public domain field to true and the fee to 0.

(4)

- 1.11 The theatres with the word 'Grand' in their name are being sold to another company. Write a query to delete all these theatres (with 'Grand' anywhere in their name) from the database.

(4)

50 marks

SECTION B OBJECT-ORIENTATED PROGRAMMING

The Swift Theatre Company owns theatres which produce local and international plays. Plays have a first date and a last date. A play will be performed at a theatre on the days between the first date and the last date (inclusive). Two different plays cannot be performed at the same theatre during the same dates. For example, if a play is performed from 1 January to 28 February, no other play can be performed at that theatre until after 28 February.

Different theatres have different numbers of seats. To promote South African theatre, the company has decided that local plays must be performed only at theatres with 200 or more seats. International plays must be performed only at theatres with fewer than 200 seats.

The type of play, local or international is represented by numbers as follows:

- 0 – Local
- 1 – International

Some theatres have trapdoors on the stage to allow for actors to appear or disappear dramatically. Therefore, some plays can only be performed in theatres with trapdoors. Plays that do not need trapdoors can be performed at any theatre.

Follow the instructions on the following pages to create a program to help the company allocate plays to theatres while adhering to all the restrictions.

Play Class

This class will represent the basic details of plays.

- **name** – the name of the play, stored as a string.
- **trap** – whether the play needs a trapdoor or not, stored as a boolean.
- **license** – the type of license, international or local, stored as an integer.
- **first** – the date of the first performance of the play, stored as a date in the format yyyy-MM-dd.
- **last** – the date of the last performance of the play, stored as a date in the format yyyy-MM-dd.

The details of the plays are stored in the file **Production.txt**. A line of the file representing the data of a play has the following format:

<name>;<trap>;<license>;<first>;<last>

For example, the first line of play details is:

Umabatha;TRUE;0;2025-03-08;2025-04-21

The play is called 'Umabatha', it does need a trapdoor. Its first performance is on 8 March 2025 and its final performance is on 21 April 2025.

Theatre Class

This class will represent the basic details of theatres.

- **name** – the name of the theatre, stored as a string.
- **trap** – whether the theatre has a trapdoor or not, stored as a boolean.
- **seats** – the number of seats the theatre has, stored as an integer.
- **tPlaysArr[]** – an array holding the plays allocated to the theatre.
- **size** – the size of the **Play** array.

The details of the theatres are also stored in the lines of the text file called **Production.txt**. A line of the file representing the data of a theatre has the following format:

<name>;<trap>;<seats>

For example:

Community Playhouse;FALSE;300

The theatre name is 'Community Playhouse', without a trapdoor, and has 300 seats.

Plays are stored on separate lines and will be allocated to the **Theatre** class at a later stage.

The first ten lines of data from the text file are shown below:

Community Playhouse;FALSE;300

Umabatha;TRUE;0;2025-03-08;2025-04-21

Boesman and Lena;FALSE;0;2025-03-04;2025-03-20

Cats;TRUE;1;2025-01-07;2025-01-20

Dramatic Acts;TRUE;160

To Kill a Mockingbird;FALSE;1;2025-01-03;2025-02-24

Frankenstien;TRUE;1;2025-03-05;2025-05-26

Tswalo;FALSE;0;2025-02-04;2025-02-28

Theatre House;FALSE;180

Have You Seen Zandile?;FALSE;0;2025-02-02;2025-02-17

NOTE: You can identify a theatre because that line of data has three pieces of information whereas a play has five.

QUESTION 2

Use the class diagram below to create a new class called **Play**. This class will be used to store the details of a play. The diagram below indicates the fields and methods that are required.

Play
Fields: - name : string - trap : boolean - license : integer - first : date - last : date + LOCAL = 0 : integer + INTERNATIONAL = 1 : integer
Methods: + Constructor(inN : string, inT : boolean, inLic : integer, inF : string, inLast : string) + getName() : string + getFirst() : date + getLast() : date + getTrap() : boolean + getLicense() : string + dateClash(inP : Play) : boolean + toString() : string

- 2.1 Create a new class named **Play** with the **name**, **trap**, **license**, **first** and **last** fields, as shown in the class diagram. These fields must not be accessible outside the class. (3)
- 2.2 Add the constants **LOCAL** and **INTERNATIONAL** to the class as shown in the class diagram. These fields must be accessible outside the class. (3)
- 2.3 Write code to create a parameterised constructor method that will accept parameters for all fields, as shown in the class diagram and assign these values to the corresponding fields. Note that **inF** and **inLast** correspond to the **first** and **last** fields and must be converted from string into a date before being assigned to the respective fields. (4)
- 2.4 Create accessor methods for the **name**, **first**, **last**, and **trap** fields. (2)
- 2.5 Code a method called **getLicense**. This method should return the string **'Local'** if the **license** field matches the **LOCAL** constant and **'International'** if the **license** field matches the **INTERNATIONAL** constant. Otherwise, it should return **'Other'**. (4)

2.6 Code a method called **dateClash**. This method accepts a **Play** parameter. This method will check whether the dates of the parameter **Play** clash with the current play's dates. The method should return true if:

- the first date of the parameter play and the current play is the same, **OR**
- the first date of the parameter play is after the first date of the current play and before the last date of the current play **OR**
- the last date of the parameter play is after the current play's first date and before the current play's last date.

Otherwise, the method should return false.

(7)

2.7 Create a **toString** method that will return the **license** (as a string), **name**, **first**, and **last** fields on the same line, separated by tabs. The format of the string should be as follows:

license<tab>**name**<tab>**first**<tab>**last**

For example:

Local Boesman and Lena 2025-03-04 2025-03-20

(2)
[25]

QUESTION 3

Use the class diagram below to create a new class called **Theatre**. This class will be used to store the details of a theatre and the plays that will be performed there.

Theatre
Fields: - name : string - trap : boolean - seats : integer - tPlaysArr[] : Play[30] - size : integer
Methods: + Constructor(inN : string, inT : boolean, inS : integer) + getName() : string + getTrap() : boolean + getSeats() : integer + addPlay(inP : Play) : boolean + toString() : string

3.1 Create a new class named **Theatre** with the **name**, **trap**, **seats**, **tPlaysArr**, and **size** fields as shown in the class diagram. These fields must not be accessible outside the class. **The tPlaysArr field must store up to 30 Play objects.** (2)

3.2 Code a parameterised constructor method that will accept parameters for the **name**, **trap**, and **seat** fields, as shown in the class diagram. Assign these values to the fields of the class. (2)

3.3 Add accessor methods for the **name**, **trap**, and **seats** fields of the class. (2)

NOTE: The **addPlay** method will be coded in a later question.

3.4 Code a **toString** method to return a string consisting of:

- The **name**, **trap**, and **seats** fields on the same line, separated by tabs.
- On the next line:
 - If the **size** field is zero, append 'No plays scheduled' to the string.
 - Otherwise, add the details of each **Play** in the **tPlaysArr** array to the string. Each play should be on a new line.

The format should be as follows:

```
name<tab>trap<tab>seats<space>'seats.'  
'No plays scheduled.'
```

For example:

```
Community Playhouse      false      300 seats.  
No plays scheduled.
```

(7)
[13]

QUESTION 4

- 4.1 Create a class called **TheatreManager**. (1)
- 4.2 Create four fields for the **TheatreManager** class as described below:
An array called **tArr** to store up to 50 **Theatre** objects.
A counter called **tSize** to count the objects added to **tArr**.
An array called **pArr** to store up to 50 **Play** objects.
A counter called **pSize** to count the objects added to **pArr**.
The fields should not be accessible outside the class. (3)
- 4.3 Create a constructor method that will read the contents of the text file **Production.txt** containing the information about **Theatres** and **Plays**. Remember that you can recognise a theatre because the line only has three parts, a play will have five parts. The method should do the following:
- Check if the file **Production.txt** exists.
 - Display a suitable error message if the file does not exist.
 - Open the file for reading.
 - Loop through the text file. In each iteration of the loop:
 - Read the line and split the data into separate parts.
 - If there are five parts, create a **Play** object and add it to the next available position in the **pArr** array and increment the **pSize** field.
 - If there are three parts, create a **Theatre** object and add it to the next available position in the **tArr** array and increment the **tSize** field.
 - Close the text file. (13)
- 4.4 Write code to create a **toString** method. This method should return a string with the details of all **theatre** objects in the **tArr** array on a new line. Use the **toString** methods created in the previous classes. (5)
- 4.5 Code a method named **sortTheatre**. This method should sort the array **tArr** based on the number of seats. The theatres should be sorted from most seats to least seats. (5)

[27]

QUESTION 5

- 5.1 Write code to create a text-based user interface called **TheatreUI** that allows simple input and output. (1)
- 5.2 Declare and instantiate a **TheatreManager** object. (1)
- 5.3 Write code to call the appropriate method in the **TheatreManager** class to sort the **theatre** array from most seats to least seats. (1)
- 5.4 Write code to call the appropriate method in the **TheatreManager** class to display a list of all the **theatre** objects. The correct output is shown below:

```
Community Playhouse    false    300 seats.
No plays scheduled.
Dinner and Drama       true     250 seats.
No plays scheduled.
Theatre House          false    180 seats.
No plays scheduled.
Dramatic Acts          true     160 seats.
No plays scheduled.
Stage and Song          true     150 seats.
No plays scheduled.
```

(1)
[4]

QUESTION 6

- 6.1 In the **Theatre** class, code the **addPlay** method as shown in the class diagram. This method should accept a **Play** object.
- The method should check whether the **Play** object parameter can be added to the theatre's **tPlaysArr** array.
 - A **Play** can only be added if the **Play** dates do not clash with any plays already in the array.
 - If the **Play** needs a trapdoor, then the theatre should have a trapdoor.
 - If the **Play** can be added, do so, and return true.
 - Otherwise, return false.
- (13)

- 6.2 In the **TheatreManager** class code a method called **allocatePlays**.
- This method should allocate every play possible to an appropriate theatre.
 - If the play has a local license, it can be allocated only to a theatre with 200 or more seats.
 - If the play has an international license, it can be allocated only to a theatre with less than 200 seats.
 - A list of any plays that cannot be allocated should be returned as a string, displayed under the heading 'Unassigned Plays'.
- (17)

- 6.3 In the **TheatreUI** class, call an appropriate method to allocate the plays to the class. Display a list of any plays that cannot be allocated. The correct output is shown below:

```
Unassigned Plays
Woza Albert
The Island
```

(1)

After allocating the plays to each theatre you may print the theatre array again to check your work. The correct allocations are shown below:

```
Community Playhouse   false   300 seats.
Local   Boesman and Lena   2025-03-04 2025-03-20
Local   Tswalo   2025-02-04   2025-02-28

Dinner and Drama     true    250 seats.
Local   Umabatha   2025-03-08   2025-04-21
Local   Have You Seen Zandile?   2025-02-02   2025-02-17

Theatre House        false   180 seats.
International   To Kill a Mockingbird   2025-01-03   2025-02-24

Dramatic Acts        true    160 seats.
International   Cats   2025-01-07   2025-01-20
International   Frankenstein   2025-03-05   2025-05-26

Stage and Song        true    150 seats.
International   Phantom of the Opera   2025-01-02
2025-01-21
```

[31]

100 marks

Total: 150 marks